Top 20 Recently Asked Playwright Scenario-Based Interview Questions & Answers

1. SSO/OAuth login takes 30 seconds. How would you avoid logging in before every test while ensuring session validity?

Answer: I would avoid performing the complete SSO login for every test because it increases execution time significantly.
I would use Playwright's **storageState** to save the authenticated browser session. I would create a separate authentication setup, perform SSO login once, save cookies/local storage/session information, and reuse that state in other tests. If the session expires, I would regenerate the authentication state.

Example:

```
await page.goto('/login');
await page.getByRole('button', { name: 'Login' }).click();
await page.waitForURL('/dashboard');

await page.context().storageState({
  path: 'auth.json'
});

// playwright.config.js
use: {
  storageState: 'auth.json'
}
```

2. Tests pass locally but randomly fail in Jenkins. How would you debug and stabilize them?

Answer: First, I would determine whether the failure is caused by timing, environment differences, network problems, browser versions, test data, parallel execution, or CI resource limitations.
I would enable traces, screenshots, videos and logs on failure, inspect the Playwright trace, compare local and Jenkins versions, and remove hard waits. I would use Playwright's auto-waiting and meaningful assertions instead of arbitrary sleeps.

Example:

```
use: {
  trace: 'retain-on-failure',
  screenshot: 'only-on-failure',
  video: 'retain-on-failure'
}
```

3. A button is visible but Playwright says “element is not stable” or “detached from DOM.” What could be the reasons?

Answer: This usually happens when the application dynamically changes the DOM. Common reasons are React re-rendering, animations, element movement, AJAX/API responses, DOM replacement, or multiple matching elements.
I would prefer a Locator because Playwright resolves the current element when the action is performed, and I would wait for meaningful application state instead of using arbitrary sleeps.

Example:

```
const submitButton = page.getByRole('button', {
  name: 'Submit'
```

```
});

await expect(submitButton).toBeVisible();
await expect(submitButton).toBeEnabled();
await submitButton.click();
```

4. The application uses virtual scrolling. How would you validate the 500th record?

Answer: Only visible rows are rendered in a virtual-scrolling table. I would scroll the relevant container until the required record becomes visible, then validate it. For large datasets, I would also consider API/database validation rather than relying only on UI scrolling.

Example:

```
const container = page.locator('.table-container');

while (!(await page.getByText('Customer 500').isVisible())) {
  await container.mouse.wheel(0, 1000);
}

await expect(page.getByText('Customer 500')).toBeVisible();
```

5. How would you automate a payment flow where the payment page opens in a new tab and contains an iframe?

Answer: I would wait for the new page using context.waitForEvent('page'), then use frameLocator() to interact with the payment iframe.

Example:

```
const newPagePromise = context.waitForEvent('page');

await page.getByRole('button', { name: 'Pay Now' }).click();

const paymentPage = await newPagePromise;

const paymentFrame =
  paymentPage.frameLocator('#payment-frame');

await paymentFrame.getByLabel('Card Number')
  .fill('4111111111111111');

await paymentFrame.getByLabel('CVV').fill('123');
```

6. The application frequently changes CSS classes after every deployment. How would you design reliable locators?

Answer: I would avoid dynamically generated CSS classes. My preferred locator strategy would be getByRole(), getByLabel(), getByPlaceholder(), getByText(), and stable data-testid attributes.
If possible, I would work with developers to add stable test attributes.

Example:

```
page.getByRole('button', { name: 'Login' });

// Preferred when supported:
page.getByTestId('login-button');
```

7. An API returns 500 intermittently. How would you continue UI automation without depending on the backend?

Answer: I would use API mocking for the unstable backend service. Playwright allows network requests to be intercepted with page.route(). I would provide controlled test data so that the UI can be tested independently, while keeping separate integration tests for the real backend.

Example:

```
await page.route('**/api/customer', async route => {
  await route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify({
      id: 101,
      name: 'Test Customer'
    })
  });
});
```

8. The same Playwright tests must run on Dev, QA, UAT and Production. How would you design environment management?

Answer: I would externalize environment-specific configuration rather than hard-code URLs. The environment would be selected through an environment variable, while sensitive credentials would be stored in CI/CD secrets.

Example:

```
const environments = {
  dev: 'https://dev.example.com',
  qa: 'https://qa.example.com',
  uat: 'https://uat.example.com',
  prod: 'https://example.com'
};

const baseURL = environments[process.env.ENV || 'qa'];
```

9. Two parallel tests update the same customer and fail intermittently. How would you make them independent?

Answer: The issue is shared state. I would generate unique test data for each test, create data during setup, clean it up afterward where appropriate, and avoid test-order dependencies. One test should not depend on another test's data or execution order.

Example:

```
const customerName =
  `Customer_${Date.now()}_${Math.random()}`;
```

10. The suite has 2,000 tests and takes 3 hours. How would you reduce execution time below 1 hour?

Answer: I would combine parallel execution, CI sharding, authentication reuse, removal of unnecessary waits, optimized test data setup, and suitable test prioritization.
For example, I could increase workers and split the suite across multiple CI machines.

Example:

```
workers: 8

# Example sharding
npx playwright test --shard=1/4
npx playwright test --shard=2/4
```

11. A download starts after clicking Export. How would you verify that the downloaded Excel/PDF contains the correct data?

Answer: I would capture the download event, save the file, verify its existence/name/extension and validate its contents. For Excel, I would parse the workbook with an appropriate Node.js library; for PDF, I would extract text and verify important values.

Example:

```
const downloadPromise = page.waitForEvent('download');

await page.getByRole('button', { name: 'Export' }).click();

const download = await downloadPromise;
await download.saveAs('report.xlsx');
```

12. The application requires OTP authentication. How would you automate the login flow without manual intervention?

Answer: For a test environment, I would prefer a test-only OTP, mock OTP service, controlled OTP retrieval, or an approved test authentication mechanism. I would coordinate with the security/development team rather than bypassing real production OTP controls.

Example:

```
// Example for a dedicated test environment
await page.getByLabel('OTP').fill('123456');

await page.getByRole('button', {
  name: 'Verify'
}).click();
```

13. How would you validate data displayed on the UI against data returned by backend APIs or the database?

Answer: I would retrieve expected data from the API or database, extract the corresponding UI values, normalize them if necessary, and compare them. This can validate the complete Database → API → UI data flow.

Example:

```
const response = await request.get('/api/customers');
const apiData = await response.json();

const uiName =
  await page.locator('.customer-name').textContent();

expect(uiName).toBe(apiData.name);
```

14. Developers complain that the automation framework is difficult to maintain. How would you redesign it?

Answer: I would separate responsibilities using Page Objects, fixtures, utilities, centralized configuration, test-data management, reporting and CI/CD integration. Tests should remain readable while reusable implementation details live in the appropriate framework layers.

Example:

```
automation/
■■■ tests/
■■■ pages/
■■■ fixtures/
```

```
■■■ utils/
■■■ test-data/
■■■ config/
■■■ reports/
■■■ playwright.config.js
■■■ package.json
```

15. The application has multiple browser windows, pop-ups, and nested iframes. How would you handle navigation between them?

Answer: I would explicitly wait for page or popup events and then use frameLocator() for iframe interactions. For nested iframes, I would chain frameLocator calls.

Example:

```
const pagePromise = context.waitForEvent('page');
await page.getByText('Open').click();
const newPage = await pagePromise;

const popupPromise = page.waitForEvent('popup');
await page.getByText('Open Popup').click();
const popup = await popupPromise;

const outerFrame = page.frameLocator('#outer-frame');
const innerFrame = outerFrame.frameLocator('#inner-frame');
```

16. A React application frequently re-renders elements, causing stale element issues. How does Playwright handle this?

Answer: Playwright's Locator API works well with dynamic applications because the locator resolves the current element when the action is performed. I would avoid relying on stale ElementHandles, use stable selectors, avoid arbitrary waits and use web-first assertions.

Example:

```
const button = page.getByRole('button', {
  name: 'Submit'
});

await expect(button).toBeVisible();
await expect(button).toBeEnabled();
await button.click();
```

17. You need screenshots, videos, traces and logs only for failed tests. How would you configure Playwright?

Answer: I would configure Playwright to retain debugging artifacts only when tests fail. This reduces storage usage while preserving the information needed for investigation.

Example:

```
use: {
  screenshot: 'only-on-failure',
  video: 'retain-on-failure',
  trace: 'retain-on-failure'
}
```

18. A test fails only in Firefox but passes in Chromium and WebKit. How would you investigate?

Answer: I would reproduce the issue specifically in Firefox, inspect trace/screenshots/logs, compare browser versions and investigate browser-specific JavaScript, CSS, timing, permissions, viewport or network behavior.
If the application itself behaves differently in Firefox, I would report it as a product/browser compatibility issue; otherwise I would make the automation logic browser-independent.

Example:

```
npx playwright test --project=firefox
```

19. How would you implement API mocking for third-party services while keeping the rest of the application real?

Answer: I would intercept only the third-party endpoint and fulfill it with controlled responses, while allowing the application's own APIs and UI to remain real. I would create mocks for success, client errors, authorization errors, server errors and timeouts.

Example:

```
await page.route(
  '**/third-party/payment/status',
  async route => {
    await route.fulfill({
      status: 200,
      contentType: 'application/json',
      body: JSON.stringify({
        status: 'success'
      })
    });
  }
);
```

20. Design a scalable Playwright automation framework from scratch for an enterprise application.

Answer: I would design the framework around scalability, maintainability, reliability and fast execution. It would include a clear folder structure, Page Object Model, reusable fixtures, utilities, test-data management, environment configuration, custom reporting, CI/CD integration, parallel execution, controlled retries and failure artifacts.

Folder structure:
automation/
■■■ tests/ → smoke, regression and API tests
■■■ pages/ → Page Object classes
■■■ fixtures/ → reusable test setup
■■■ utils/ → API, DB, file and data utilities
■■■ test-data/ → controlled test data
■■■ config/ → environment configuration
■■■ reports/ → generated reports
■■■ playwright.config.js

I would use storageState for authentication reuse, stable Locator-based selectors, unique test data, parallel workers and CI sharding. I would use retries carefully so they do not hide genuine defects. Screenshots, videos and traces would be retained on failure.

The CI/CD flow would typically be:
Git Push → Install Dependencies → Run Validation → Smoke Tests → Regression Tests → Generate Report → Publish Artifacts.

My main goal would be to keep tests reliable, independent, maintainable and fast.

Example:

```
use: {
  screenshot: 'only-on-failure',
  video: 'retain-on-failure',
  trace: 'retain-on-failure'
}

retries: process.env.CI ? 2 : 0

reporter: [
  ['html'],
  ['junit', { outputFile: 'results.xml' }]
]
```

Jai yadav
QA engineer